

Data Lives Here: Databases & SQL

2026-04-29 · lively date Hase

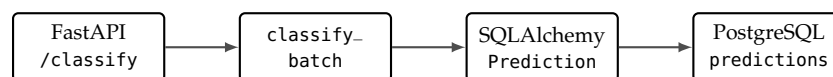
YOUR API WORKS. NOW MAKE IT REMEMBER.

The Why

Block 5 left us with a service that turns drawings into predictions over HTTP. The classifier loads at startup, the API validates each request, the systemd unit keeps it running. What it does not do is remember anything. The moment the response leaves the server, the prediction is gone.

ACROSS TEN LECTURE BLOCKS WE BUILD PIXELWISE. This block is the first that adds persistent state. Every block from here forward, the analytics in Block 9, the feature flag in Block 8, the frontend results panel in Block 7, depends on the database schema you design today.

WHY PERSIST AT ALL? A response that vanishes the moment it is sent is a response that cannot be counted, compared, or queried. A database makes the prediction permanent: every drawing becomes a row, the class label becomes a column, the confidence becomes another, and the model version becomes a third. Block 9's analytics endpoint reads from this table; Block 8's feature flag relies on the version field; the frontend in Block 7 fetches recent results to show in the UI.



PIXELWISE now has an HTTP service that classifies digits. The `GET /results` endpoint from Block 5 is a stub returning an empty list, with a note pointing at this block. This block fills that stub by wiring the API to a PostgreSQL database.

BEYOND THIS COURSE, the same Figure 1 will be produced from SQL retaining complete predictions with wrong predictions highlighted. The prediction of SQL is removed, and not visible any more there, predictions only in the steps with having returning the response.

Hands On Experience

CONSIDER THIS SCENARIO: a teammate asks how confident the model has been on average across the last hundred predictions. You

realise nobody can answer that question. The API returned the confidence to every caller, but nothing recorded it, and “the last hundred predictions” is not a thing your service can talk about because it never wrote them down. The only way back to those numbers is to ask the users to draw their digits again, which is not an option.

THE SCHEMA IS A CONTRACT, the same way the API contract is a contract. Get it right and every later question, distribution per digit, average confidence per model version, error rate over time, becomes a query. Get it wrong and you find out later, with a hundred thousand rows already written, that the column you need does not exist. This block introduces migrations as the tool that lets the schema evolve without losing the rows already in it.

THIS SIXTH BLOCK introduces persistence for PixelWise. By the end you will have

- written your first SQL queries against a PostgreSQL database,
- modelled the predictions table as a SQLAlchemy class,
- wired POST /classify so every prediction becomes a row,
- replaced the GET /results stub with a real query,
- run your first schema migration with Alembic to add a missing column,
- and configured database credentials through .env.

THE COST OF FORGETTING. Anything not stored is not analysable. A model running for a month without persistence has produced a month of data that can never be recovered. The decision to write the database schema now is the decision to make the next month answerable.

DESIGNING SCHEMAS is its own discipline. We deliberately start with a near-trivial schema, three columns, and grow it through a migration in the same session. The point is to feel the cycle of design, deploy, regret, migrate, which is how every real database evolves.

Relational Databases and SQL

A RELATIONAL DATABASE STORES DATA IN TABLES. Each table has a fixed schema: a list of named columns, each with a type. Each row is a single record. The schema is the contract that every row obeys; the database refuses inserts that violate it.

SQL IS THE LANGUAGE you use to talk to a relational database. Four operations cover almost everything:

```
-- Insert a row
INSERT INTO predictions (prediction, confidence,
                        model_version)
VALUES ('7', 0.94, 'v1');

-- Read rows
SELECT prediction, confidence FROM predictions
    WHERE model_version = 'v1'
    ORDER BY created_at DESC
    LIMIT 10;

-- Update a row
UPDATE predictions SET confidence = 0.95
    WHERE id = 42;

-- Delete a row
DELETE FROM predictions WHERE id = 42;
```

THE FOUR OPERATIONS ARE WHAT EVERY WEB APPLICATION DOES TO ITS DATABASE ALL DAY LONG: write a row when something happens, read rows back when somebody asks, update a row when state changes, delete one when it should be gone. Every higher-level pattern, an ORM call, a migration, an analytics query, reduces to a sequence of these.

PostgreSQL over SQLite

SQLITE IS CONVENIENT. The database is a single file, there is no server to install, no users, no network configuration. For a single-developer prototype or a desktop application, this is exactly what you want.

VOCABULARY TO KEEP STRAIGHT. A *table* is a named set of rows; a *row* is one record; a *column* is a named field with a type; a *primary key* is the column (or set of columns) that uniquely identifies each row. Most tables use an auto-incrementing integer or a UUID as their key.

WHAT YOU WILL MEET LATER. JOIN combines rows from multiple tables; GROUP BY, COUNT, and AVG aggregate; ORDER BY and LIMIT shape the output; CREATE INDEX speeds up queries that scan large tables. The full treatment belongs in a dedicated database course; the four operations above are enough to get PixelWise running.

WHERE SQLITE STOPS WORKING. SQLite has no user authentication, no password protection, and limited concurrency: a single writer blocks all readers. For a multi-user web service where 30 students hit the same database at the same second, the file-lock model becomes the bottleneck. The same file is also harder to back up while the application is running.

POSTGRESQL IS A PRODUCTION DATABASE. It runs as a separate process, accepts connections over the network, has real users with real permissions, and supports concurrent reads and writes without serialising the whole file. `psql` is the operator's interface, the same shape of tool as `systemctl` from Block 5.

SETTING UP THE DATABASE on the server is two commands inside `psql`. You log in as the `postgres` superuser to create a dedicated user and database for the application:

```
sudo -u postgres psql
```

```
-- inside psql:
CREATE USER pixelwise WITH PASSWORD 'secret';
CREATE DATABASE pixelwise OWNER pixelwise;
\q
```

CONNECT AS THE APPLICATION USER to confirm the credentials work:

```
psql -U pixelwise -d pixelwise -h localhost
```

The prompt that comes back is what your service will see when it connects through SQLAlchemy in the next section. If `psql` cannot reach the database, neither can the API; debug here first, before any Python is involved.

PostgreSQL: <https://www.postgresql.org/>
 PostgreSQL Docs: <https://www.postgresql.org/docs/current/>
 SQLite: <https://www.sqlite.org/>

WHY A DEDICATED USER? The `postgres` superuser can drop any database, create any user, and read any table. A dedicated `pixelwise` user can only do what its privileges allow, which by default is read and write its own tables. The principle is the same as the non-root producer from Block 1: the account the application uses is the smallest one that works.

Exercise: PostgreSQL on the Server

CATCHING UP VIA THE REFERENCE TAG. The course maintains a public reference repository at <https://github.com/schutera/pixelwise> with a tag at the end of every block. v0.4 marks the end of Block 5; v0.5 will mark the end of this block. If your repository has drifted, reset to v0.4 before starting:

```
git fetch origin
git reset --hard v0.4
```

INSTALL POSTGRESQL ON prod. With the theory above as the why, the how is two commands. On the prod VM, install the server and start it:

```
ssh produser@192.168.56.11
sudo apt update
sudo apt install -y postgresql
sudo systemctl status postgresql
```

CREATE A DEDICATED USER AND DATABASE:

```
sudo -u postgres psql -c \
    "CREATE USER pixelwise WITH PASSWORD 'secret';"
sudo -u postgres psql -c \
    "CREATE DATABASE pixelwise OWNER pixelwise;"
```

VERIFY THE CONNECTION with the application user:

```
psql -U pixelwise -d pixelwise -h localhost
```

If psql fails to connect, fix the credentials before going any further; SQLAlchemy will report the same error in a less direct way. A working psql is the foundation of everything the rest of the block builds on.

EXERCISES sit next to the theory they exercise. Each pair takes you from concept to running command on the same topic, and the snapshot from Block 1 is your safety net if a step goes wrong.

UPDATE setup-server.sh. The provisioning script from Block 3 should now install PostgreSQL too. Add postgresql to the apt install line, commit, and push. A fresh VM provisioned from the script after this change has the database ready without you typing a second command.

SQLAlchemy: Tables as Python Classes

SQLALCHEMY IS A PYTHON LIBRARY that maps database tables to Python classes. You define a Prediction class with fields like prediction, model_version, and created_at; SQLAlchemy generates the SQL to create the table, insert rows, and query data. This is called an ORM, an object-relational mapper, and is standard in Python web development.

THE MODEL CLASS declares the columns and their types in plain Python:

```
from sqlalchemy import (Column, Integer, String,
                        DateTime)
from sqlalchemy.orm import declarative_base
from datetime import datetime

Base = declarative_base()

class Prediction(Base):
    __tablename__ = "predictions"

    id = Column(Integer, primary_key=True)
    prediction = Column(String, nullable=False)
    model_version = Column(String, nullable=False)
    created_at = Column(DateTime,
                        default=datetime.utcnow())
```

AN ENGINE AND A SESSION are the runtime objects that connect the model to the database:

```
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker

engine = create_engine(os.getenv("DATABASE_URL"))
SessionLocal = sessionmaker(bind=engine)

# create the table on first run
Base.metadata.create_all(engine)
```

INSERTING A ROW from inside the API handler is now a few lines:

```
db = SessionLocal()
```

SQLAlchemy: <https://www.sqlalchemy.org/>
 SQLAlchemy Tutorial: <https://docs.sqlalchemy.org/en/20/tutorial/>

WHY AN ORM? Two reasons. The first is ergonomic: writing Python is faster than writing string SQL, and the editor knows the field names. The second is safety: SQLAlchemy uses parameterised queries by default, which means user input never ends up concatenated into a SQL statement. SQL injection becomes a footgun the framework refuses to hand you.

THE CONNECTION STRING is a single URL:
 postgresql://user:pass@host/dbname.
 It encodes everything the driver needs to reach the database, and it lives in .env for the same reasons the API key did in Block 5. A leaked DATABASE_URL is a leaked database.

```
db.add(Prediction(prediction=result["prediction"],  
                  model_version="v1"))  
db.commit()
```

The handler still calls `classify_batch` from Block 4 and returns `ClassifyResponse` from Block 5; the only addition is one round-trip to the database before the response leaves the server.

Exercise: The Prediction Model

TURN THE THEORY ABOVE INTO CODE. On dev, install the driver and the ORM into the virtual environment:

```
source .venv/bin/activate
pip install sqlalchemy psycopg2-binary
pip freeze > requirements.txt
```

WRITE `app/models.py` with the minimal schema, only the class label and the model version:

```
from sqlalchemy import (Column, Integer, String,
                        DateTime)
from sqlalchemy.orm import declarative_base
from datetime import datetime

Base = declarative_base()

class Prediction(Base):
    __tablename__ = "predictions"
    id = Column(Integer, primary_key=True)
    prediction = Column(String, nullable=False)
    model_version = Column(String, nullable=False)
    created_at = Column(DateTime,
                        default=datetime.utcnow)
```

ADD `DATABASE_URL` to `.env` and `.env.example`:

```
DATABASE_URL=postgresql://pixelwise:secret@localhost/pixelwise
```

CREATE THE TABLE ONCE from a small initialisation script:

```
from sqlalchemy import create_engine
from app.models import Base
import os
from dotenv import load_dotenv

load_dotenv()
engine = create_engine(os.getenv("DATABASE_URL"))
Base.metadata.create_all(engine)
```

Run it on prod. Confirm with `psql` that `\dt` lists the predictions table.

THE SCHEMA IS INTENTIONALLY TOO SMALL. The first version stores only the predicted class and the model version. Confidence is missing on purpose; you will feel the gap in the next exercise and use Alembic to fix it. The point is to feel the migration cycle, not to skip past it.

Exercise: Wiring the API to the Database

UPDATE `app/main.py` so `POST /classify` writes a row before returning the response. The handler from Block 5 stays the same shape; one new dependency creates a database session per request:

```
from app.models import Prediction, SessionLocal

@app.post("/classify",
         response_model=ClassifyResponse,
         dependencies=[Depends(verify_api_key)])
def classify(req: ClassifyRequest):
    arr = np.array(req.pixels,
                   dtype=np.uint8)[np.newaxis]
    result = classify_batch(arr)[0]

    db = SessionLocal()
    db.add(Prediction(
        prediction=result["prediction"],
        model_version="v1"))
    db.commit()
    db.close()

    return result
```

REPLACE THE `GET /results` STUB with a real query:

```
@app.get("/results")
def results():
    db = SessionLocal()
    rows = (db.query(Prediction)
            .order_by(Prediction.created_at.desc())
            .limit(20).all())
    db.close()
    return {"results": [
        {"id": r.id,
         "prediction": r.prediction,
         "model_version": r.model_version,
         "created_at": r.created_at.isoformat()}
        for r in rows]}
```

RESTART THE SERVICE AND FEED IT DATA. Twenty `curl` requests later, `GET /results` returns twenty rows. The stub from Block 5 is now complete; the system stores what it predicts.

ONE SESSION PER REQUEST. A session is the unit of database work. Open it at the start of the handler, close it at the end. The framework patterns FastAPI documents (`Depends` for sessions) do this more elegantly; the manual version above is the simplest one that shows what is happening.

ASK THE QUESTION the schema cannot answer: "how confident is the model on average?" The API returns confidence to every caller, but the database only stored the class label. You cannot answer the question from the data on disk. That is the gap the next exercise closes.

Schema Migrations with Alembic

SCHEMAS ALWAYS CHANGE. You design the table, deploy it, store 500 predictions. Then you realise you forgot a column. You cannot drop the table and recreate it; those 500 rows are real data. You need a tool that alters the live schema without losing them.

ALEMBIC is the migration tool for SQLAlchemy. It generates migration scripts from changes you made to the model, applies them in order, and tracks which migrations have run on which database.

```
# Initialise Alembic in the repo
alembic init alembic
```

```
# After you change app/models.py:
alembic revision --autogenerate \
    -m "add confidence column"
```

```
# Apply the migration
alembic upgrade head
```

IN A REAL SYSTEM with millions of rows and live traffic, you cannot just drop and recreate. Migrations are how schemas evolve safely, in production, with the application running. This topic is covered in much more depth in dedicated database and backend courses; here we run one migration end to end so the cycle of edit-model, generate-migration, apply-migration is concrete.

Alembic: <https://alembic.sqlalchemy.org/>

THE METAPHOR. A migration is a recipe that describes a change to the schema and how to roll it back. The migration tool keeps an internal log of which migrations have run, applies new ones in order, and refuses to run the same one twice. The result is that the schema in the database always matches the migrations folder in Git.

THE AUTOGENERATE IS A DRAFT, NOT A FINAL ANSWER. Alembic compares the model in code against the schema in the database and guesses what changed. For straightforward additions it gets it right; for renames or splits it needs a human to read the generated script and adjust it. Always read the migration before you run it.

SIDENOTE: REDIS. Redis is an in-memory key-value store with sub-millisecond reads. It is the right tool for caching, session storage, and rate-limit counters. For PixelWise, every drawing is unique pixel data, so the cache hit rate would be effectively zero; we mention it so you recognise it later. Redis: <https://redis.io/>

Exercise: First Migration with Alembic

INITIALISE ALEMBIC IN THE REPO. With the migration cycle from the theory above in mind, configure the tool to use the same engine as the application:

```
pip install alembic
pip freeze > requirements.txt
alembic init alembic
```

Edit `alembic.ini` so `sqlalchemy.url` reads from `.env`, and edit `alembic/env.py` to import `Base` from `app.models` so autogenerate sees your tables.

ADD THE MISSING COLUMN to the model in `app/models.py`:

```
from sqlalchemy import Float

class Prediction(Base):
    __tablename__ = "predictions"
    id = Column(Integer, primary_key=True)
    prediction = Column(String, nullable=False)
    confidence = Column(Float)          # NEW
    model_version = Column(String, nullable=False)
    created_at = Column(DateTime,
                              default=datetime.utcnow)
```

GENERATE AND APPLY THE MIGRATION:

```
alembic revision --autogenerate \
    -m "add confidence column"
alembic upgrade head
```

VERIFY WITH `psql`:

```
psql -U pixelwise -d pixelwise -h localhost \
    -c "SELECT prediction, confidence
        FROM predictions
        ORDER BY created_at DESC LIMIT 5;"
```

Old rows show `NULL` for confidence; new predictions store the real number. That gap between old and new rows is the visible signature of a migration: the column exists everywhere, but its values are populated only from the moment the new code shipped.

READ THE GENERATED CONFIG. `alembic init alembic` writes a folder of starter files. Skim them once before you change anything; the structure is small and the mental model becomes obvious by reading the templates.

READ THE MIGRATION BEFORE YOU RUN IT. The generated script lives in `alembic/versions/`. Open it, confirm the only operation is `add_column("confidence")`, and only then run `upgrade head`. A wrong migration on a populated table is much harder to undo than to prevent.

WIRE CONFIDENCE INTO THE HANDLER:

```
db.add(Prediction(
    prediction=result["prediction"],
    confidence=result["confidence"],
    model_version="v1"))
```

Restart the service. Draw a digit; confirm the new row in psql has both fields populated. Commit, push, and the repository now matches v0.5:

```
git add app/main.py app/models.py alembic/ \
    alembic.ini requirements.txt .env.example
git commit -m "Add PostgreSQL persistence and first migration"
git push
```

A NOTE ON WHAT COMES NEXT. The system is reachable on the network and remembers what it does, but nobody outside the SSH tunnel can use it. The next block puts a frontend and an Nginx reverse proxy in front of the API, and PixelWise becomes accessible from a browser at a real URL.

Self-Reflection and Recap

SELF-REFLECTION questions to guide your thoughts during and after the exercises:

- Why do we store predictions in a database rather than just returning them in the API response?
- What is the difference between dropping and recreating a table versus running a migration?
- Why did the old rows show NULL for the confidence column after the migration?
- What happens if you store the database password directly in your Python code instead of in `.env`?
- Why is a dedicated database user with minimal privileges better than connecting as the superuser?
- How does the ORM make SQL injection harder, and where does the protection break down?

RECAP of key concepts:

- Relational databases store data in typed tables; SQL is the language that queries and changes them.
- PostgreSQL is the production database for this course; SQLite is too limited for a multi-user web service.
- SQLAlchemy maps tables to Python classes and uses parameterised queries by default.
- Migrations let schemas change without losing rows; Alembic generates and applies them.
- Database credentials live in `.env`, never in code; the application user has minimal privileges.

MILESTONE. Every digit prediction is stored in PostgreSQL with its confidence and model version. `GET /results` returns real data from the table; the Block 5 stub is gone. You have run your first schema migration on a live database with Alembic and watched old rows coexist with new ones. The next chapter places Nginx and a browser frontend in front of the API so PixelWise becomes a website.

SECURITY THREAD. DB credentials live in `.env`, never in code. The dedicated `pixelwise` user has no superuser rights and cannot create databases. SQLAlchemy's parameterised queries prevent SQL injection by default; never build queries with string formatting.

TEASER. The service stores predictions and returns them, but only somebody with `curl` and the API key can ask. What does it take to put this in front of a real browser at a real URL?